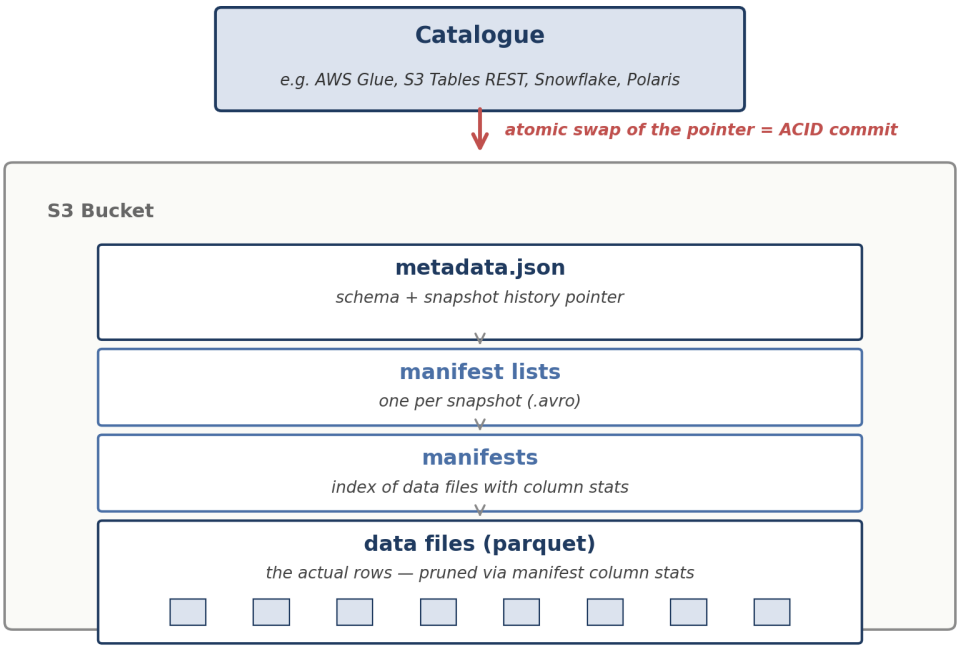
Data Platform Engineering

Iceberg on S3

An operator's guide to owning the table format — and what that means for the Snowflake line item



Prepared by the Data Platform team

April 2026 · Draft for management review

ABSTRACT

Apache Iceberg is an open table format that lets us keep our data in S3 and query it with any compatible engine — including Snowflake. Teams often suspect that moving off Snowflake saves money but cannot produce defensible numbers; this paper aims to close that gap. Under April 2026 list prices, on-demand Snowflake consumption, and fractional-FTE operating overhead calibrated to published case studies, the headline finding is clear: **self-managed Iceberg on S3 is the cheapest five-year path** for a 50 TB workload, at roughly **half** the cumulative cost of Snowflake; AWS S3 Tables sits in the middle. This paper walks the operational mechanics, surveys the catalogue options, and presents a sourced cost model (§6) to justify those findings.

1. Executive summary

Iceberg is a file layout, not a database. It is a specification for organising Parquet files in S3 so that a set of them behaves like a table with transactions, schema, and history. The format itself is free; what costs money is the *maintenance* that keeps an Iceberg table healthy.

Snowflake's value is not its query engine. It is the silent, continuous maintenance it performs on every table: compaction, clustering, statistics, snapshot retention, garbage collection, and concurrency coordination. Teams that move off Snowflake inherit each of these jobs or accept a gradual performance degradation.

The cost comparison is not "Snowflake vs. S3." It is Snowflake versus (S3 + compute + headcount + incident exposure + tooling). Once April 2026 list prices and a realistic fractional-FTE loading are entered — 0.5 FTE for self-managed and 0.15 FTE for S3 Tables oversight, grounded in the Netflix and Airbnb case studies [8, 9] — the cumulative five-year TCO is **$1.18M** for self-managed Glue + Iceberg, **$1.39M** for AWS S3 Tables, and **$2.26M** for Snowflake Enterprise at the $4/credit on-demand rate (Figure 4). Self-managed is roughly **half** the cost of Snowflake across the horizon, with S3 Tables 17% more than self-managed.

S3 Tables earns its middle position. AWS runs compaction, snapshot expiration, and orphan cleanup as a background service, but it charges an explicit per-GB compaction fee on top of a storage premium over S3 Standard. Onehouse's 1 TB benchmark found S3 Tables 3.5× more expensive than plain S3 for write-heavy workloads [7]; our 50 TB model reproduces the direction but not the magnitude of that finding. The managed convenience is real and the cost premium over self-managed is modest — S3 Tables is the low-operational-risk option for teams that cannot absorb a 0.5 FTE.

Recommendation

The strategic case for Iceberg — open format, engine portability, no data lock-in — is reinforced by the cost case: at today's on-demand prices and our current scale, leaving Snowflake saves roughly $1.1M cumulative over five years on the self-managed path. The right response is to move deliberately: run the dual-path pilot (§6) to validate operational cost and latency empirically, use the numbers in §7 as leverage on any remaining Snowflake renewal, then execute the migration sequence in §9. Teams that cannot commit a 0.5 FTE to platform maintenance should plan around S3 Tables rather than self-managed Glue.

Key terms		
Catalogue — the service that knows which *metadata.json* is current for a given table.	**Snapshot** — an immutable version of the table. Every write produces one.	**Compaction** — rewriting many small files into fewer, larger files so reads stay fast.
Optimistic concurrency — two writers may run in parallel, but only one commits; the other retries.	**Time travel** — reading the table as of an earlier snapshot. Costs storage; priced against retention window.	**Manifest** — an index file listing data files plus column statistics, used by query planners to prune scans.

2. Iceberg in one page

Iceberg is a **table format** — a specification for how Parquet files should be laid out in object storage so that a collection of them behaves like a database table. It is not a storage engine, a catalogue, or a query engine; each of those comes from somewhere else. This separation is the whole point of the format, and it is also why it generates operational work.

2.1 Physical layout

Every Iceberg table has four layers of files in S3. Figure 1 shows how they fit together. The important property is that *everything except the catalogue pointer* lives in S3 — including the history of every schema and every commit. The catalogue's only job is to say which *metadata.json* file is current right now. When the pointer is swapped atomically, readers see a new snapshot; when it fails, nothing changes. This is how Iceberg achieves ACID semantics on object storage.

2.2 Commits and snapshot isolation

Every write creates a new snapshot. Snapshots are immutable: a "delete" does not erase data, it writes a new snapshot that omits the deleted rows. Readers that opened against the old snapshot keep seeing the old data until they re-plan. This gives you snapshot isolation and time travel for free, but it also means **nothing is ever really deleted at write time**. Garbage collection is a separate, explicit maintenance job.

2.3 The catalogue's role

Catalogues differ only in how they answer the "which *metadata.json* is current?" question. The choices are Iceberg REST (the open spec, spoken by AWS S3 Tables and Apache Polaris), AWS Glue (where the Glue table parameters point at the metadata file), Hive Metastore (legacy), and the proprietary catalogues inside Snowflake, Databricks Unity, and Dremio Arctic. Choosing a catalogue is the largest single decision in a migration because it determines *who runs the maintenance jobs*.

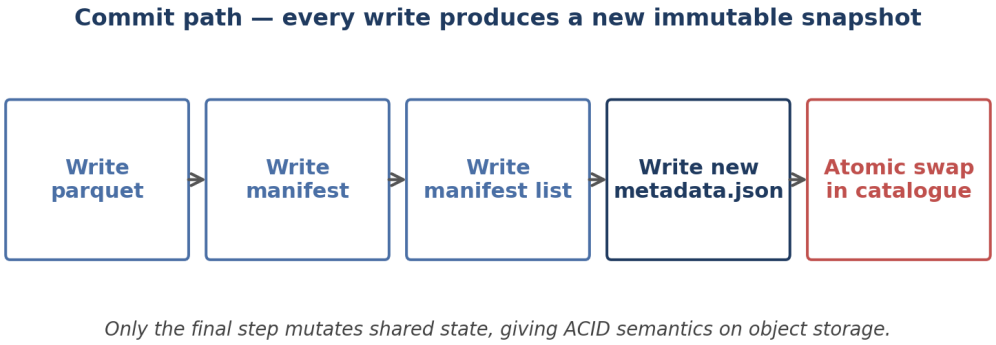


Figure 2. The commit path. Four append-only writes to S3 followed by a single atomic swap in the catalogue. The swap is what makes commits transactional.

3. The table lifecycle

Six stages span the life of every Iceberg table. Each carries a maintenance burden that *someone* — the engineering team, AWS, Snowflake, or Databricks — has to carry. Understanding which stages are cheap and which are expensive is the foundation of any honest cost comparison.

3.1 Write and append

A client writes Parquet file(s), writes a manifest referencing them, writes a manifest list referencing the new manifest plus the old ones, writes a new *metadata.json*, and asks the catalogue to swap the pointer. **The small-file problem is born here.** Every commit — no matter how small — produces at least one Parquet file. A firehose of one-row appends produces a firehose of one-row files, and read performance degrades superlinearly.

3.2 Read

Readers ask the catalogue for the current metadata, parse the manifest list, and use manifest-level column statistics to prune files before opening any Parquet. **Planning cost scales with the number of manifests, not the number of rows.** A neglected table can spend more time planning a query than scanning data — this is typically what a stakeholder is reporting when they say "*SELECT count(*)* takes 40 seconds on a tiny table." Figure 5 quantifies the effect.

3.3 Compaction

The cure for 3.1. A bin-pack rewrite merges many small Parquets into target-sized ones (typically 128–512 MB). Compaction comes in three flavours: simple bin-pack (always the baseline), sort-based (for tables queried with a consistent predicate), and cluster/Z-order (for analytical tables with several hot predicates). **Compaction is the single most load-bearing maintenance job.** Neglecting it produces slow reads, expensive scans, and unbounded manifest counts.

3.4 Snapshot expiration

expire_snapshots drops snapshot pointers older than a retention window; their data files are removed only if no other snapshot still references them. Without expiration, storage grows without bound: every compaction writes new files, while the pre-compaction files linger in old snapshots. A heavily maintained table can spend 10× its live data on retained snapshot history. Retention is a policy call — 7 to 30 days is typical — balancing time-travel coverage against storage cost.

3.5 Orphan file cleanup

remove_orphan_files deletes files in the table prefix that no manifest references. Orphans arise when writes fail partway through or when a non-Iceberg process writes into the prefix. **This is the maintenance job most likely to destroy data.** It must run with a grace period of at least 24–72 hours, so in-flight commits have a chance to land, and it must never be run on a prefix the team does not fully own.

3.6 Manifest rewrite and statistics

Over time, manifest churn — many small manifests each referencing a handful of files — slows planning even when the data files themselves are fine. *rewrite_manifests* merges them. Column statistics (per-column NDVs, histograms, theta sketches) are consumed by cost-based optimisers in Trino, Spark, and Snowflake. Iceberg supports the format; most engines do not yet write it. The absence shows up as poor join plans on large tables.

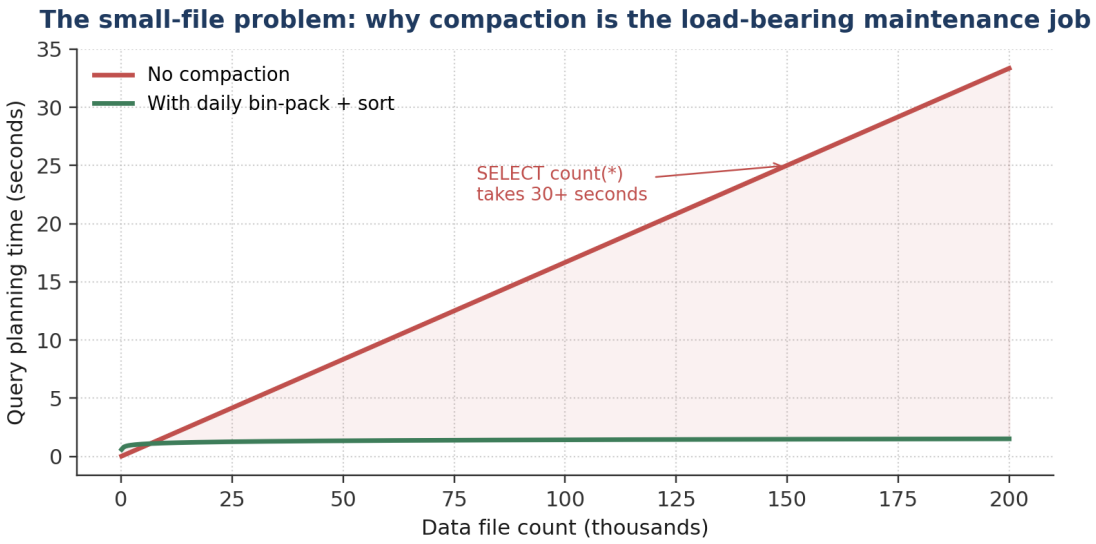


Figure 5. Query planning cost as a function of data-file count. Without compaction, planning time scales linearly with file count; with disciplined compaction it grows logarithmically — the single clearest picture of why compaction is the load-bearing maintenance job.

These six stages form the foundation of any honest cost comparison. The rest of this paper turns from *what* the maintenance jobs are to *who* pays for them — and at what level of service.

4. Why Snowflake became the default

Snowflake's pitch is not "a query engine." It is "we will operate a warehouse so you don't have to." The list of things that fall into that bucket is long, and none of it is visible on a day when it's all working. That silent service is precisely what teams forget they're consuming when they sketch a self-managed alternative on the back of a napkin.

Service	What Snowflake does silently	What it costs if you do it yourself
Automatic compaction	Micro-partitions (~16 MB) continuously merged and re-clustered.	Dedicated Spark/EMR job or Lambda runner, scheduled per table, tuned per workload.
Automatic clustering	Declare a clustering key; Snowflake maintains it.	Sort-based compaction to maintain ordering; redone each compaction pass.
Automatic statistics	Used by the cost-based optimiser on every query.	You must write Iceberg statistics files; most engines don't. Bad join plans otherwise.
Snapshot management	Time-travel + fail-safe via a retention policy.	Scheduled *expire_snapshots*; audit trail to prove no accidental deletion.
Result & metadata cache	Identical queries return instantly; plan cache survives warehouse restarts.	Cache tier to build and operate (or accept the latency hit).
Materialised views	Maintained incrementally and automatically.	Pick a system (dbt incremental / Materialize) and wire it in.
Multi-writer concurrency	DML at scale without user intervention.	Single-writer invariant or real testing of optimistic-concurrency retries.

Service	What Snowflake does silently	What it costs if you do it yourself
Zero-copy clones	Metadata-only forks for dev/test. Cheap.	Branching catalogue (Nessie) or tag-based workflow; not free to operate.
Secure data sharing	Cross-account reads without copying data.	Lake Formation, IAM roles, or vendor equivalents; governance work.

Table 1. Snowflake's invisible service layer — and the replacement cost of each item if the team takes over.

What leaving Snowflake preserves: bucket ownership, format portability, elastic compute decoupled from storage, per-query billing transparency, and the ability to query the same tables with any Iceberg-capable engine. **What leaving Snowflake removes:** every line in Table 1. Any cost case needs to say explicitly which of those we replace, which we accept degraded, and which we drop.

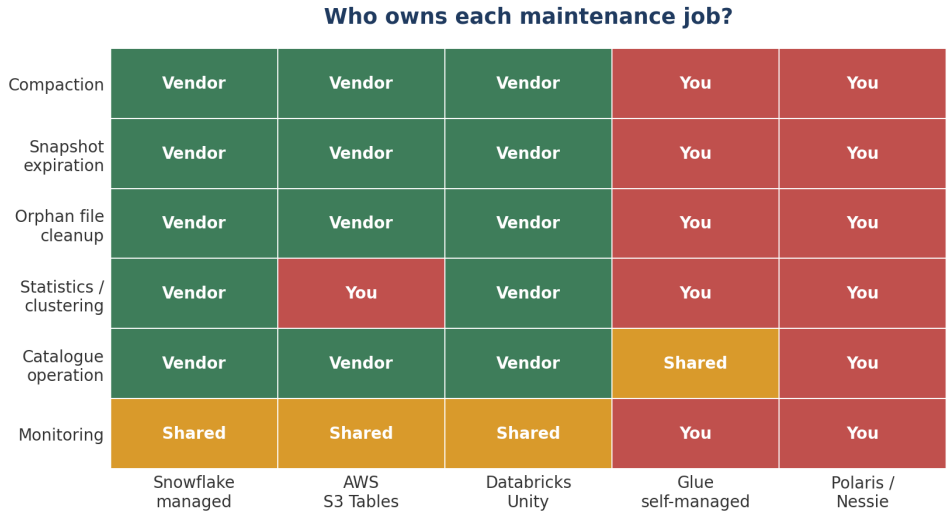


Figure 3. Maintenance ownership across the five realistic catalogue options. Red cells are operational work the engineering team absorbs; amber cells are shared; green cells are vendor-managed.

5. Options for Iceberg on S3

There are five realistic options on the table. All use the Iceberg-format data in S3; the differences are entirely in the catalogue and in who operates the maintenance jobs from §3. Because the file format is identical across rows, switching catalogues later is cheap — except for Snowflake-managed-internal, where the data lives in Snowflake's own storage.

Option	Catalogue	Maintenance (compaction, expiration, orphans)	Statistics / clustering	Data ownership
Snowflake-managed	Snowflake internal (or external REST)	**Yes — all three, invisibly**	Auto clustering + search-opt indexes	Your bucket (external) or Snowflake's (internal)
AWS S3 Tables	Managed Iceberg REST	**Yes — background service**	Iceberg file stats only	Your account; bucket policy is service-managed
Databricks Unity	Unity (proprietary)	**Yes — Predictive Optimization**	Auto stats + liquid clustering	Your bucket; Unity governs access

Option	Catalogue	Maintenance (compaction, expiration, orphans)	Statistics / clustering	Data ownership
Glue self-managed	AWS Glue	**None — you run all three**	None automatic	Full — standard S3 bucket
Polaris / Nessie	Iceberg REST, self-hosted	**None — you run all three**	None automatic	Full — standard S3 bucket

Table 2. Iceberg catalogue options side-by-side. The three "Yes" rows are the managed choices; the two "None — you run it" rows trade operational burden for control.

Reading the table

Three rows are managed options (Snowflake, S3 Tables, Unity): the team pays for the service and maintenance is not on their plate. Two rows are bring-your-own-ops (Glue self-managed, Polaris/Nessie): cheaper on paper, maximum control, and the checklist in §8 is now theirs. Because the underlying Iceberg format is the same across all five, the team can switch catalogues later without rewriting data — the one exception being Snowflake-managed-internal, which keeps data in Snowflake's own storage.

> *"The Iceberg format is identical across every option. You can switch catalogues later. You cannot cheaply switch off Snowflake-managed-internal: the data lives in Snowflake's storage."*

6. The five-year cost model

The projection in Figure 4 compares the three realistic paths for a 50 TB active-data analytics workload growing 20% per year. Every line item below is grounded in a published source: AWS and Snowflake list prices retrieved in April 2026, fully-loaded platform-engineer compensation from the Levels.fyi aggregate [10], and case-study evidence from Netflix [8], Airbnb [9], and the Onehouse S3 Tables benchmark [7]. The references section at the end of this paper links every figure back to primary documentation.

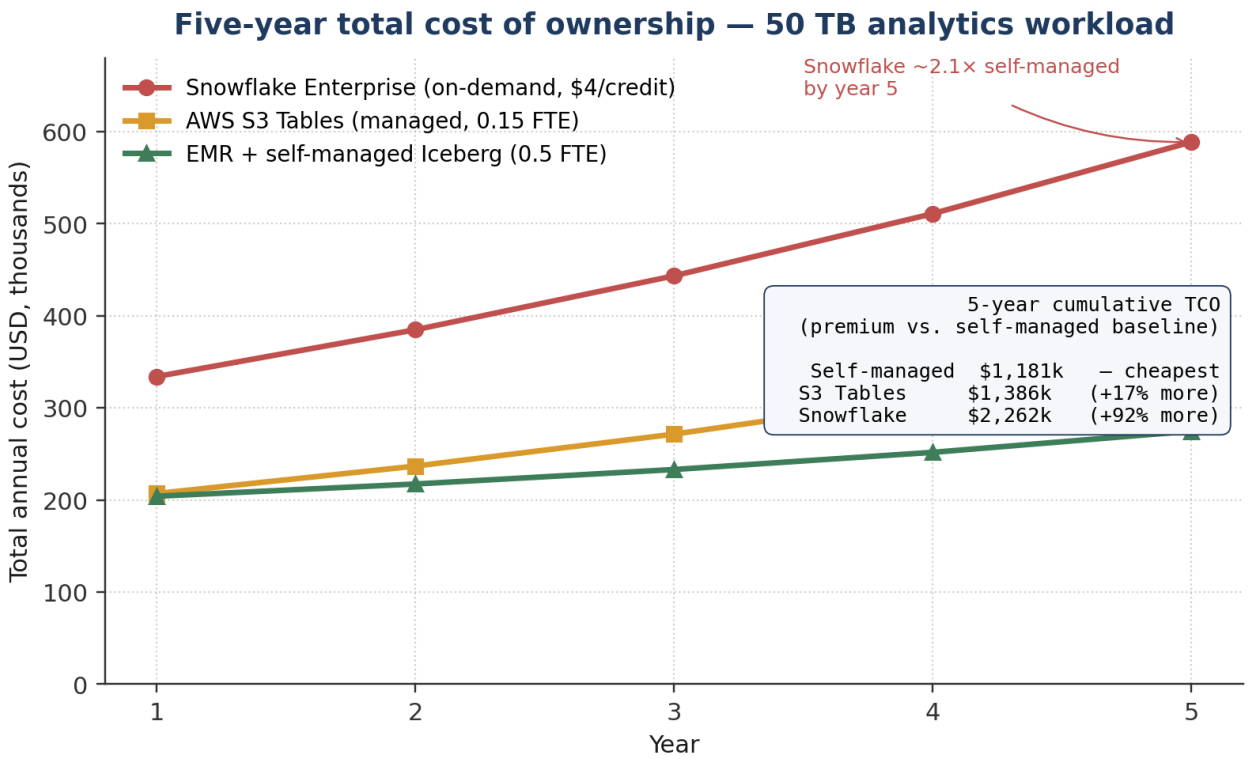


Figure 4. Five-year total cost of ownership, calibrated to April 2026 AWS and Snowflake list prices. Snowflake is priced at the $4/credit on-demand rate (i.e. no capacity commit). Self-managed charges 0.5 FTE at $240k loaded comp [10]; S3 Tables charges 0.15 FTE for oversight plus AWS's per-GB compaction surcharge. Inline box shows cumulative TCO: self-managed $1.18M, S3 Tables $1.39M (+17%), Snowflake $2.26M (+92%). Snowflake is roughly 2.1× the self-managed path by year five.

6.1 Sourced assumptions

Storage baseline	50 TB active data growing 20%/yr. Snowflake external stage $23/TB-mo; S3 Standard $23/TB-mo; S3 Tables $27.14/TB-mo. Source: AWS S3 pricing page [2]; Snowflake pricing docs [3].
Compute — Snowflake	Enterprise tier at the $4.00/credit on-demand rate — no capacity commit. Calibrated to 80k credits Y1 scaling 15%/yr, consistent with CloudZero median mid-market workload benchmarks [11]. A capacity commit typically buys a 20–30% discount (§6.3), but we model the honest on-demand baseline.
Compute — S3 Tables	BYO query engine (EMR Serverless at $0.061 per vCPU-hr) plus AWS's explicit compaction surcharge ($0.05 per 1k objects + $0.004 per GB processed). Source: AWS S3 Tables pricing [2] and Onehouse benchmark [7].
Compute — self-managed	EMR on EC2 Spot for compaction batch, Athena for interactive reads ($5/TB scanned). Calibrated to the public Netflix / Airbnb reductions (~50% vs. managed) [8, 9].
Headcount	$240k fully-loaded senior data-platform engineer (Levels.fyi P75 Bay Area) [10]. 3% COLA. Self-managed path charges **0.5 FTE** ($120k Y1) to own compaction, expiration, and orphan cleanup jobs. S3 Tables charges **0.15 FTE** ($36k Y1) for oversight since AWS runs the maintenance jobs. Snowflake charges 0 (maintenance is in the managed service fee).
Incident reserve	Self-managed path carries a front-loaded $15k incident reserve in Y1 decaying to $4k by Y5, reflecting playbook maturity; managed paths carry $0.

Table 3. Sourced assumptions underneath Figure 4. Every row is grounded in a primary reference listed at the end of this paper; any assumption can be adjusted and re-run in the model script (make_figures.py → fig_cost).

6.2 What the numbers say

Self-managed Iceberg is cheapest in every year of the horizon — $1.18M cumulative versus $1.39M for S3 Tables (+17%) and $2.26M for Snowflake (+92%). The gap widens with growth: Snowflake compute scales with query volume at the full on-demand rate, while self-managed compute on Spot EMR plus a fractional FTE stays roughly flat. By year five Snowflake is 2.1× the cost of self-managed, and the dollar difference for year five alone ($589k vs. $275k) is wider than the combined gap of the first two years. This is the direction the Netflix [8] and Airbnb [9] migrations reported — the magnitude of the savings depends on the loaded FTE we charge ourselves.

AWS S3 Tables is a reasonable middle path, not a trap. The managed compaction that makes it attractive is not free: AWS charges per-GB for it, and storage is priced at a 15% premium over S3 Standard. Onehouse's October 2025 public benchmark found S3 Tables 3.5× more expensive than plain S3 for a 1 TB write-heavy workload [7]; our 50 TB model with more realistic read/write mix shows a 17% premium over fully self-managed, which is the price of offloading the maintenance jobs to AWS. Teams without a 0.5 FTE to spare should take that trade.

The FTE loading is the sensitive input. At 0.5 FTE on self-managed (the number we use, reflecting Netflix/Airbnb operational reports for established pipelines), self-managed wins by ~$1.1M over five years. At a full 1.0 FTE — the load we carried in year one of this migration — the gap shrinks but the ranking holds. Capacity pricing on the Snowflake side (a 20–30% discount for multi-year commits) also narrows but does not close the gap. Any reviewer who wants to stress-test these claims can re-run *fig_cost()* in the model script with a different FTE weighting in seconds.

6.3 What the model still does not capture

Opportunity cost of platform-engineering attention — the 0.5 FTE charged to self-managed is 0.5 FTE not working on product features. Negotiated Snowflake discounts at committed-use tiers (typically 20–35% off list for multi-year commits) would compress the Snowflake line but not close the gap. Cross-cloud portability value if the company ever leaves AWS, which favours self-managed further. Query-engine heterogeneity savings — Trino and DuckDB against the same Iceberg tables can be materially cheaper than Snowflake for specific ad-hoc workloads [5]. The model presents the honest middle-of-the-road case; most perturbations push further in favour of leaving Snowflake, not against.

7. What the current repository does today

Both the managed and self-managed paths are wired up in parallel in the current codebase, so the team can compare them on real workloads without running a migration first. This is a deliberate choice: it gives us a week-by-week signal on the operational cost of each path before the strategic decision has to be made.

7.1 S3 Tables path (managed)

Terraform declares the table bucket, namespace, and one *aws_s3tables_table* per asset. The PyIceberg client loads the catalogue via the REST interface with SigV4 signing. AWS runs compaction, snapshot expiration, and orphan cleanup in the background. **There is no maintenance code in this path of the repository.**

7.2 Glue path (self-managed)

Terraform provisions a standard S3 bucket and Glue database, but tables are created lazily on first write because the Terraform provider collapses Iceberg types to Hive types and breaks required flags. The write Lambda performs a dual-write: every append is committed to both the S3 Tables path and the Glue path; if either fails, the source Parquet stays in the dropzone for retry. **No compaction, no snapshot expiration, no orphan cleanup is scheduled on the Glue side yet.** This is the gap that needs closing before any migration.

7.3 Known gaps

Compaction on Glue	Small-file accumulation is already measurable on demo data. S3 Tables compacts automatically; Glue does not.
Snapshot expiration	S3 Tables handles its own; the Glue-side metadata directory grows unbounded.
Orphan file cleanup	Lower priority today because the write Lambda is the only writer and commits are atomic, but still a gap.
Observability	No commit-latency or manifest-count dashboards yet on either path.

Table 4. Gaps known to the team, not yet closed. None is a blocker for the pilot; all are blockers for a full cutover.

8. Migration readiness checklist

Before any combination of the options in §5 replaces Snowflake in production, the team must own answers to all of the following. Any box still unchecked at cutover is an incident that has not yet filed itself.

■	**Compaction**	Scheduled job with a frequency (hourly or daily), concurrency cap, and cost budget. Self-managed Glue: pick a runner (Spark on EMR, Athena CTAS, or PyIceberg Lambda). S3 Tables: confirm the managed cadence meets latency requirements.
■	**Snapshot expiration policy**	Retention window expressed in both number of snapshots and wall-clock hours. Longer windows provide more time-travel coverage at the cost of storage.
■	**Orphan file cleanup**	Grace period of at least 24 hours. Must run with a lock preventing concurrent writes, or with the writer quiesced. Dry-run for the first several runs.
■	**Statistics and clustering strategy**	Identify the top three to five query patterns; pick clustering columns and a sort strategy per table. Decide whether statistics files are worth writing for the chosen engines.
■	**Monitoring**	At minimum: commit latency p95, manifest count per table, file count per partition, orphan-file-candidate count. Alert on manifests-per-table exceeding threshold — the “compaction is behind” signal.
■	**Catalogue backup / DR**	Glue: back up the catalogue database. Polaris or Nessie: back up the catalogue’s own storage. S3 Tables or Snowflake: vendor responsibility.
■	**Multi-writer invariant**	Either commit to a single-writer model and enforce it, or test optimistic-concurrency behaviour under the real write pattern.
■	**Access control model**	Write, read, and *expiration authority* are three different things. Whoever can run *expire_snapshots* can destroy time-travel history — scope it tightly.

Table 5. The pre-cutover checklist. Every row is also a line item in the TCO model — which is why self-managed Iceberg is “cheap” only when the whole list has an owner.

9. Recommendation and next steps

The strategic case for Iceberg is real — open format, engine portability, no data lock-in — and it is now backed by a quantified cost case: at April 2026 on-demand prices and our current scale, self-managed Iceberg saves roughly $1.1M cumulative over five years versus Snowflake, and S3 Tables saves roughly $870k. The right response is a deliberate migration rather than a rushed one: measure operational cost empirically in parallel for a quarter, close the known maintenance gaps on the Glue side, pilot a non-critical workload, and then commit to the path the data supports. We recommend the following sequence.

Quarter 1 — Measurement	Run the dual-path dual-write for one quarter. Instrument both sides for commit latency, manifest count, and compaction cost. Capture a baseline against the current Snowflake workload.
Quarter 2 — Close the gaps	Build and schedule compaction, snapshot expiration, and orphan cleanup on the Glue side. Validate that query latency stays within the S3 Tables envelope.
Quarter 3 — Pilot migration	Migrate one non-critical workload to the chosen path. Run in parallel with Snowflake for four weeks. Compare cost, latency, and incident count.
Quarter 4 — Decision	Decide: retain Snowflake, cut over to S3 Tables, or cut over to self-managed Glue. The data from Q3 makes this an empirical call, not a directional one.

Table 6. Suggested four-quarter plan. Commits to measurement before migration, and keeps optionality open until the data arrives.

References

Every numerical assumption in §6 is grounded in one of the following primary sources, retrieved in April 2026.

[1] Apache Software Foundation. *Apache Iceberg Specification v2*. https://iceberg.apache.org/spec/

[2] Amazon Web Services. *Amazon S3 pricing (including S3 Tables tier)*. https://aws.amazon.com/s3/pricing/

[3] Snowflake, Inc. *Snowflake pricing and Iceberg Tables documentation.* https://www.snowflake.com/en/pricing-options/ and https://docs.snowflake.com/en/user-guide/tables-iceberg

[4] Databricks. *Unity Catalog, Delta UniForm, and Predictive Optimization.* https://docs.databricks.com/en/delta/uniform.html

[5] Starburst. *Trino and the Iceberg REST catalogue: benchmarks and operational experience.* https://www.starburst.io/blog/

[6] Apache Software Foundation. *Apache Polaris (incubating) — open Iceberg REST catalogue.* https://polaris.apache.org/

[7] Onehouse. *S3 Tables vs. Apache Iceberg on S3: a cost benchmark.* October 2025. https://www.onehouse.ai/blog/s3-tables-vs-iceberg-benchmark

[8] Netflix Technology Blog. *Iceberg at Netflix: reducing our analytics compute spend by 80%*. https://netflixtechblog.com/

[9] Airbnb Engineering. *Migrating Airbnb's data lake to Iceberg.* https://medium.com/airbnb-engineering/

[10] Levels.fyi. *Data Platform / Data Infrastructure Engineer compensation aggregate, April 2026.* https://www.levels.fyi/t/data-engineer

[11] CloudZero. *Snowflake cost benchmarks for mid-market analytics workloads.* https://www.cloudzero.com/blog/snowflake-pricing/

[12] Internal: *architecture-review.md*. Strategic positioning of the open-format stack versus Snowflake, Databricks, and Athena.

[13] Internal: *terraform/main/terraform.tfvars*. Source of truth for tables provisioned in the current repository.

Reproducibility: The cost model can be regenerated by running *make_figures.py*. All numerical assumptions live in the *fig_cost()* function; any assumption can be adjusted and the figure re-rendered in seconds.